

Exception handling recommendation while working with throws :

```
public static void main(String [] args)
{
    try
    {
        m1();
    }
    catch(X e){}
}

public static void m1() throws CheckedException
{
    m2();
}

public static void m2() throws CheckedException
{
    //Any checked exception
}
```

Note : It is strongly recommended that while working with checked exception, method uses throws keyword to skip from the current situation and try to send the exception object to the CALLER method then any of the caller method must handle the exception by using try catch otherwisw program will be terminated abnormally.

Some important rules regarding the checked Exception :

a) If the try block does not throw any checked exception then in the corresponding catch block we can't handle checked exception.It will generate compilation error i.e "exception never thrown from the corresponding try statement"

Example :-

```
public class Test
{
    public static void main(String[] args)
    {
        try
        {
            //try block is not throwing checked exception
            //i.e. InterruptedException
        }
        catch (InterruptedException e) //error
        {
        }
    }
}
```

Note :- The above rule is not applicable for Unchecked Exception

```
    try
    {

    }
    catch(ArithmeticException e) //Valid
    {
        e.printStackTrace();
    }
}
```

b) If the try block does not throw any exception then in the corresponding catch block we can write Exception OR Throwable because both are the super classes for all types of Exception whether it is checked or unchecked.

Throwable = Exception + Error (Unchecked)  
Exception = Direct Sub classes of Exception + RuntimeException (Unchecked)

```
package com.ravi.checked_exception_rules;

public class CatchingWithSuperClass
{
    public static void main(String[] args)
    {
        try
        {

        }
        catch(Throwable e) //Exception OR Throwable both are allowed
        {
            e.printStackTrace();
        }
    }
}
```

c) At the time of method overriding if the super class method does not reporting or throwing checked exception then the overridden method of sub class not allowed to throw checked exception otherwise it will generate compilation error but overridden method can throw Unchecked Exception.

```
package com.ravi.checked_exception_rules;

import java.io.IOException;
```

```
class Super
{
    public void show()
    {
    }
}
```

```
class Sub extends Super
{
    @Override
    public void show() //throws IOException
    {
    }
}
```

```
public class MethodOverridingWithChecked {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }

}
```

d) If the super class method declare with throws keyword to throw a checked exception, then at the time of method overriding, sub class method may or may not use throws keyword to throw an exception.

If the Overriden method is also using throws keyword to throw checked exception then it must be either SAME exception class or SUB class, it should not be SUPER class as well as we can't add more exceptions in the overridden method.

```
package com.ravi.checked_exception_rules;

import java.io.EOFException;
import java.io.IOException;
```

```
class Parent
{
    public void show() throws IOException
    {
    }
}
```

```
class Child extends Parent
{
    @Override
    public void show() //Valid
    {
        IO.println("Exception type must be same type OR sub type");
    }
}
```

```
public class MethodOverridingWithThrows
{

    public static void main(String[] args)
    {

    }

}
```

e) Just like return keyword, we can't use throw keyword inside static and non static block to throw an exception object because all initializers must be executed normally.

```
public class ExceptionDemo
{
    static
    {
        throw new ArithmeticException(); //CE
    }

    public static void main(String []args)
    {

    }

}
```

We can use throw keyword in the protection of try-catch so the code will be executed normally.

```
public class Main
{
    public static void main(String[] args)
    {

    }

    static
    {
        try
        {
            throw new ArithmeticException();
        }
        catch (ArithmeticException e)
        {
            System.out.println("Exception Handled");
        }
    }

}
```

f) If we call any method and if the method throws any checked exception OR java.lang.Exception OR java.lang.Throwable then handling is compulsory at caller Method otherwise it will generate compilation error because Exception and Throwable both are representing checked and Unchecked.

```
public class Main
{
    public static void main(String[] args)
    {
        m1(); //Handling is compulsory
    }

    public static void m1() throws Throwable //OR Exception
    {
        [Error]
    }

}
```

Nested try block :

If we write a try block inside another try block then it is called Nested try block.

```
try //Outer try
{
    statement1;
    try //Inner try
    {
        statement2;
    }
    catch(Exception e) //Inner catch
    {
    }
}
catch(Exception e) //Outer Catch
{
}
```

The execution of inner try block depends upon outer try block that means if we have an exception in the Outer try block then inner try block will not be executed.

```
NestedTry.java
-----
package com.ravi.basic;
```

```
public class NestedTryBlockDemo
{
    public static void main(String[] args)
    {
        try
        {
            String str = "123";
            IO.println("String length is :"+str.length());
            try
            {
                int no = Integer.parseInt(str);
                IO.println("Value is :"+no);
            }
            catch(NumberFormatException e)
            {
                IO.println("Number is not in a proper format");
            }
        }
        catch(NullPointerException e)
        {
            IO.println("Ref variable is pointing to null");
        }
    }
}
```

The execution of inner try block depends upon outer try block that means if we have an exception in the Outer try block then inner try block will not be executed.

Writing try-catch inside catch block :

We can write try-catch inside catch block but this try-catch block will be executed if the catch block will executed that means if we have an exception in the try block.

We can also write try-catch inside finally block.

```
package com.ravi.basic;

import java.util.InputMismatchException;
import java.util.Scanner;

public class TryWithCatchInsideCatch
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try(sc)
        {
            System.out.print("Enter your Roll number :");
            int roll = sc.nextInt();
            System.out.println("Your Roll is :"+roll);
        }
        catch(InputMismatchException e)
        {
            System.err.println("Provide Valid input!!");

            try
            {
                System.out.println(10/0);
            }
            catch(ArithmeticException e1)
            {
                System.err.println("Divide by zero problem");
            }
        }

        finally
        {
            System.out.println("Finally block");
            try
            {
                Object []obj = new Integer[3];
                obj[0] = 12;
                obj[1] = 45;
                obj[2] = "90"; //Invalid
            }
            catch(ArrayStoreException e)
            {
                System.err.println("Invalid element so can't store in array");
            }
        }
    }
}
```

try-catch with return statement

If we write try-catch block inside a method and that method is returning some value then we should write return statement in both the places i.e inside the try block as well as inside the catch block.

We can also write return statement inside the finally block only (Not recommended), if the finally block is present. After this return statement we cannot write any kind of statement. (Unreachable but finally block should be executed completely)

Always finally block return statement having more priority then try-catch return statement because finally block return statement override try - catch return statement so compiler will generate warning.

```
package com.ravi.return_example;

public class ReturnDemo1 {

    public static void main(String[] args)
    {
        IO.println(accept());
    }

    public static int accept()
    {
        try
        {
            IO.println("Try block");
            int x = 10/0;
            return 100;
        }
        catch(ArithmeticException e)
        {
            IO.println("Catch block");
            return 200;
        }
        //IO.println("Unreachable");
    }

}
```

```
package com.ravi.return_example;

public class ReturnDemo2 {

    public static void main(String[] args)
    {
        IO.println(accept());
    }

    public static int accept()
    {
        try
        {
            IO.println("try");
            return 10/0;
        }
        catch(ArithmeticException e)
        {
            IO.println("catch");
            return 20;
        }
    }

}
```

```
package com.ravi.return_example;

public class ReturnDemo2 {

    public static void main(String[] args)
    {
        IO.println(accept());
    }

    public static int accept()
    {
        try
        {
            IO.println("try");
            return 10/0;
        }
        catch(ArithmeticException e)
        {
            IO.println("catch");
            return 20;
        }
    }

}
```

```
package com.ravi.return_example;

public class ReturnDemo3 {

    public static void main(String[] args)
    {
        IO.println(accept());
    }

    @SuppressWarnings("finally")
    public static int accept()
    {
        try
        {
            IO.println("try");
            return 100;
        }
        catch(ArithmeticException e)
        {
            IO.println("catch");
            return 200;
        }
        finally
        {
            IO.println("finally");
            return 300;
        }
    }

}
```

Initialization of a local variable in try and catch :

A local variable must be initialized inside try block as well as catch block OR at the time of declaration.

If we initialize inside the try block only then from catch block we cannot access local variable value, Here initialization is compulsory inside catch block also.

```
package com.ravi.basic;

public class VariableInitialization
{
    public static void main(String[] args)
    {
        int x;
```

```
        try
        {
            int []jarr = new int[-10];
            x = 190;
            System.out.println(x);
        }
        catch(Exception e)
        {
            x = 200;
            System.out.println(x);
        }
    }
}
```